

IBM8910 – PROGRAMAÇÃO PARA ANÁLISE DE DADOS

Guia de Estudos

Projeto Extensionista · Etapa I — Preparação e transformação dos dados

Passo a passo comentado, com a base *Melbourne Housing Snapshot* (melb_data.csv) como exemplo

POR QUÊ

Este guia acompanha, passo a passo, o notebook **Etapa_I_Projeto_GrupoX.ipynb**. Para cada procedimento exigido na Parte I você encontra: o que é, por que fazer, o código comentado linha a linha, o resultado esperado, os erros mais comuns e o que escrever no relatório. Os números citados foram obtidos executando o notebook de verdade sobre a base de exemplo.

Sumário

Como usar este guia	2
Passo 0 — Preparar o ambiente	3
Passo 7.1 — Leitura da base de dados	3
Passo 7.2 — Seleção e exclusão de colunas	5
Passo 7.3 — Renomeação das colunas	6
Passo 7.4 — Identificação e tratamento dos tipos de dados	8
Passo 7.5 — Variáveis de data e hora	9
Passo 7.6 — Dados ausentes	11
Passo 7.7 — Tratamento e padronização dos valores	14
Passo 7.8 — Criação de novas variáveis	15
Passo 7.9 — Discretização de variáveis	17
Passo 7.10 — Variáveis dummy	19
Passo 7.11 — Estrutura final do dataframe	20
Passo 8 — Tabelas de frequência	21
Passo 9 — Visualização dos dados	23
Passo 10 — Exportação da base preparada	28
Montando e entregando a Parte I	29
Guia rápido de erros e mensagens	30
Glossário	31
Exercícios de fixação	32
Respostas dos exercícios	33

Como usar este guia

A Parte I do projeto pede que o grupo mostre que sabe **ler, entender, limpar e transformar** uma base de dados real antes de analisá-la. Nada disso é difícil isoladamente; o desafio é fazer cada passo com critério e saber explicar o porquê. É exatamente isso que será avaliado: o código é apenas o meio.

Como estudar

1. Abra o notebook ao lado deste guia e execute **uma célula por vez**, na ordem. Antes de rodar, leia a explicação do passo aqui.
2. Depois de rodar, compare a sua saída com a seção **“Saída esperada”**. Se for diferente, releia o código e os erros comuns do passo.
3. Ao final de cada passo, leia a caixa **“No relatório”**: ela diz o que precisa aparecer no seu PDF (código, trecho do resultado, explicação e justificativa).
4. Faça os exercícios do final do guia sem olhar as respostas.

A base de exemplo e a base do seu grupo

O notebook usa dados públicos de vendas de imóveis em Melbourne (Austrália). A **sua base será outra**, e por isso as decisões mudam: quais colunas excluir, como tratar ausentes, qual variável discretizar. O que se repete é o **método**. Em cada passo há uma caixa **“Como decidir no seu projeto”** com as perguntas que você deve responder sobre a sua própria base.

Visão geral dos passos

Passo	O que se faz	Principais comandos
0	Preparar o ambiente	<code>import pandas, numpy, matplotlib</code>
7.1	Ler a base	<code>pd.read_csv, shape, head</code>
7.2	Excluir colunas	<code>drop</code>
7.3	Renomear colunas	<code>rename</code>
7.4	Ajustar tipos de dados	<code>dtypes, astype</code>
7.5	Converter datas	<code>pd.to_datetime</code>
7.6	Tratar dados ausentes	<code>isna, fillna, groupby + transform</code>
7.7	Padronizar valores e corrigir erros	<code>str.strip, loc, duplicated</code>
7.8	Criar variável numérica e categórica (.loc)	<code>operações entre colunas, loc</code>
7.9	Discretizar uma variável contínua	<code>pd.qcut / pd.cut</code>
7.10	Criar variáveis dummy	<code>pd.get_dummies</code>
7.11	Conferir a estrutura final	<code>shape, dtypes</code>
8	Tabelas de frequência	<code>value_counts</code>
9	Gráficos	<code>matplotlib (hist, boxplot, bar, scatter)</code>
10	Exportar a base preparada	<code>to_csv(index=False)</code>

POR QUÊ

Por que esta ordem? Cada passo prepara o terreno do seguinte. Renomear antes de tudo facilita escrever o resto do código; corrigir tipos antes de tratar ausentes evita surpresas (por exemplo, um número lido como texto); tratar ausentes e erros *antes* de criar variáveis garante que as novas colunas (como preço por m²) não herdem valores vazios ou absurdos; e discretizar e criar dummies só faz sentido sobre dados já limpos.

Passo 0 — Preparar o ambiente

O que é

É a primeira célula do notebook: importar as bibliotecas que serão usadas e definir o endereço da base.

Como fazer

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option("display.max_columns", None)
```

Código	O que acontece
<code>import pandas as pd</code>	Carrega o pandas , biblioteca de tabelas (DataFrames). O apelido <i>pd</i> é uma convenção universal.
<code>import numpy as np</code>	Carrega o numpy , usado aqui sobretudo para representar valores ausentes (<i>np.nan</i>).
<code>import matplotlib.pyplot as plt</code>	Carrega o matplotlib , usado nos gráficos da seção 9.
<code>pd.set_option("display.max_columns", None)</code>	Pede ao pandas que mostre todas as colunas ao exibir uma tabela (por padrão ele resume com "...").

ATENÇÃO

Execute as células em ordem e não repita células que modificam o **DataFrame**. Rodar duas vezes a célula que exclui colunas gera *KeyError* (a coluna já não existe); rodar duas vezes a que cria dummies duplica colunas. Se algo ficar inconsistente, use **Reiniciar kernel e executar tudo** (Restart & Run All). Esse é o teste final de que o seu notebook funciona do zero.

DICA

Se estiver no Google Colab, as bibliotecas acima já vêm instaladas. Em um computador local, instale com `pip install pandas numpy matplotlib`.

Passo 7.1 — Leitura da base de dados

O que é

Carregar o arquivo para a memória como um **DataFrame** do pandas: uma tabela em que cada linha é um registro (aqui, um imóvel vendido) e cada coluna é um atributo.

Por que fazer

Todo o resto depende de uma leitura correta. Se o separador, o formato decimal ou a codificação de caracteres estiverem errados, os erros se propagam em silêncio: números viram texto, acentos viram símbolos estranhos e colunas se fundem.

Como fazer

```
URL = "https://raw.githubusercontent.com/DeepFluxion/2026_2_IBMEC_PROG_ANALISE_DADOS/
      refs/heads/main/datasets/melb_data.csv" # (na prática, em uma linha só)

df = pd.read_csv(URL)

print(f"Quantidade de linhas: {df.shape[0]}")
print(f"Quantidade de colunas: {df.shape[1]}")
df.head()
```

Código	O que acontece
<code>URL = ...</code>	Guarda o endereço do arquivo em uma variável. Também funciona com um caminho local, como <code>"dados/melb_data.csv"</code> .
<code>df = pd.read_csv(URL)</code>	Lê o CSV e cria o DataFrame df . Por padrão espera vírgula como separador e ponto como decimal.
<code>df.shape</code>	Devolve uma tupla (<i>linhas, colunas</i>). <code>shape[0]</code> é o número de linhas e <code>shape[1]</code> o de colunas.
<code>f"...{df.shape[0]}"</code>	<i>f-string</i> : insere o valor da expressão dentro do texto.
<code>df.head()</code>	Mostra as 5 primeiras linhas, para você olhar os dados e confirmar que a leitura fez sentido.

O que você deve ver

SAÍDA ESPERADA

Quantidade de linhas: 13580 Quantidade de colunas: 21
--

Seguido de uma tabela com as primeiras linhas (Suburb, Address, Rooms, Type, Price...). Se o `shape` for `(13580, 1)` ou os nomes das colunas vierem todos juntos, há problema de separador.

Parâmetros úteis do `read_csv`

Parâmetro	Quando usar	Exemplo
<code>sep</code>	Arquivo separado por ponto e vírgula, tabulação etc. (comum em CSV exportado do Excel em português)	<code>sep=";"</code>
<code>decimal</code>	Números com vírgula decimal (1234,56)	<code>decimal=","</code>
<code>encoding</code>	Acentos aparecendo como "Ã£", "Ã§", ou erro <code>UnicodeDecodeError</code>	<code>encoding="latin-1"</code>
<code>thousands</code>	Números com separador de milhar (1.234.567)	<code>thousands="."</code>
<code>usecols / nrow</code>	Ler só algumas colunas ou só as primeiras linhas (arquivos enormes)	<code>usecols=["price"], nrow=1000</code>

Exemplo típico de CSV brasileiro: `pd.read_csv("vendas.csv", sep=";", decimal=",", encoding="latin-1")`.

Verificações extras recomendadas

- `df.info()`: tipos, quantidade de valores não nulos e uso de memória por coluna.
- `df.describe()`: resumo estatístico das colunas numéricas (mínimo, máximo, média, quartis). Já revela valores suspeitos.
- `df.duplicated().sum()`: quantas linhas são repetidas (na base original: 0).

Erros comuns

Sintoma	Causa provável	Correção
Uma única coluna enorme com tudo junto	Separador diferente de vírgula	Usar <code>sep=";"</code> (ou o separador correto)
Números aparecem como texto (objeto/str)	Vírgula decimal ou símbolos como R\$	<code>decimal=","</code> e limpar o texto antes de converter (passo 7.4)
UnicodeDecodeError ou acentos quebrados	Codificação diferente de UTF-8	<code>encoding="latin-1"</code> ou <code>"cp1252"</code>
FileNotFoundError	Caminho errado ou arquivo não enviado ao Colab	Conferir o caminho e a pasta atual do notebook
HTTP Error 404 ao ler pela URL	Endereço digitado incorretamente	Copiar a URL completa do arquivo "raw"

NO RELATÓRIO

Código da leitura; quantidade de linhas e colunas; as primeiras linhas (*head*); se foi preciso usar *sep*, *decimal* ou *encoding* — qual problema você viu e como ficou depois do ajuste. Se nenhum parâmetro foi necessário, diga isso.

Passo 7.2 — Seleção e exclusão de colunas

O que é

Remover as colunas que não serão úteis ao projeto, mantendo apenas o que ajuda a responder às suas questões de investigação.

Por que fazer

- **Reduz ruído e complexidade:** menos colunas, menos chance de erro e análises mais claras.
- **Identificadores únicos** (endereço, CPF, nome) não servem para agrupar nem comparar. Além disso, dados pessoais exigem cuidado (LGPD).
- **Colunas redundantes** repetem a mesma informação de outra coluna e podem distorcer análises estatísticas.

Como fazer

```
colunas_excluidas = ["Address", "SellerG", "Bedroom2"]
df = df.drop(columns=colunas_excluidas)

print("Colunas removidas:", colunas_excluidas)
print(f"Quantidade de colunas após a exclusão: {df.shape[1]}")
```

Código	O que acontece
<code>colunas_excluidas = [...]</code>	Lista com os nomes (exatamente como estão na base) das colunas a remover. Listar antes de excluir facilita documentar.
<code>df.drop(columns=...)</code>	Devolve um novo DataFrame sem essas colunas. Ele não altera o original se você não guardar o resultado.
<code>df = ...</code>	Reatribui o resultado a <i>df</i> . Sem isso, nada muda (um erro muito comum).

As três exclusões do exemplo

Coluna	Justificativa
Address	Identificador de altíssima cardinalidade: 13.378 valores distintos em 13.580 linhas. Não ajuda em análises agregadas.
SellerG	Nome da imobiliária/corretor (268 valores distintos). Não é objeto de nenhuma das 3 questões de investigação.
Bedroom2	Redundante com <i>Rooms</i> : correlação de 0,94 e valores idênticos em 95% das linhas. Manter as duas duplicaria a informação.

O que você deve ver

SAÍDA ESPERADA

Colunas removidas: ['Address', 'SellerG', 'Bedroom2']
Quantidade de colunas após a exclusão: 18

COMO DECIDIR NO SEU PROJETO

Antes de excluir uma coluna da **sua** base, pergunte:

- Ela ajuda a responder alguma das minhas 3 questões de investigação?
- É um identificador único (código, nome, documento, endereço)? Contém dado pessoal sensível?
- Repete a informação de outra coluna (por exemplo, “estado” e “sigla do estado”)?
- Tem um único valor para todas as linhas (não varia, logo não informa nada)?
- Tem quase tudo ausente e sem como recuperar?

Erros comuns

- **KeyError: "[Address]" not found in axis**: a coluna já foi removida (célula executada duas vezes) ou o nome está escrito diferente da base.
- **Excluir uma coluna que será usada depois** (por exemplo, a variável de preço). Confira o dicionário de dados antes.
- **Esquecer de justificar**. O relatório pede nome e justificativa de cada coluna excluída.

NO RELATÓRIO

Tabela com o **nome de cada coluna excluída e a justificativa**, mais o código do *drop* e a nova quantidade de colunas.

Passo 7.3 — Renomeação das colunas

O que é

Trocar os nomes das colunas por nomes padronizados, claros e fáceis de digitar no código.

Boas práticas

Prática	Exemplo (ruim → bom)
Letras minúsculas	BuildingArea → building_area
snake_case: palavras separadas por _ , sem espaços	Year Built → year_built
Sem acentos nem caracteres especiais (ç, ~, %, /)	Preço (R\$) → preco_reais
Nomes descritivos e consistentes (mesmo idioma)	Regionname → region_name
Corrigir erros de grafia da origem	Lattitude → latitude
Evitar nomes que colidam com atributos do pandas (count, min, T, size)	Usar df["coluna"] ao invés de df.coluna

Como fazer

```
mapa_colunas = {
    "Suburb": "suburb",      "Rooms": "rooms",      "Type": "type",
    "Price": "price",       "Method": "method",    "Date": "date",
    "Distance": "distance", "Postcode": "postcode", "Bathroom": "bathroom",
    "Car": "car",           "Landsize": "landsize", "BuildingArea": "building_area",
    "YearBuilt": "year_built", "CouncilArea": "council_area",
    "Latitude": "latitude",  "Longitude": "longitude",
    "Regionname": "region_name", "Propertycount": "property_count",
}
df = df.rename(columns=mapa_colunas)
```

Código	O que acontece
mapa_colunas = {...}	Dicionário nome antigo: nome novo . Escrevê-lo explicitamente deixa o “antes e depois” registrado no código.
df.rename(columns=...)	Aplica o mapa. Colunas que não estão no dicionário permanecem como estão.
df = ...	Mais uma vez: é preciso reatribuir, pois <i>rename</i> devolve um novo DataFrame.

DICA

Atalho para bases grandes: `df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")` limpa todos os nomes de uma vez. Depois use *rename* só para os casos especiais (acentos, erros de grafia).

O que você deve ver

SAÍDA ESPERADA

```
Colunas antes: ['Suburb', 'Rooms', 'Type', 'Price', 'Method', 'Date', 'Distance', ...]
Colunas depois: ['suburb', 'rooms', 'type', 'price', 'method', 'date', 'distance', ...]
```

Erros comuns

- **Chave do dicionário escrita errada:** o pandas *ignora* em silêncio a coluna que não encontra. Para ser avisado, use `df.rename(columns=mapa, errors="raise")`.
- **Esquecer o *df* =:** nada é renomeado.
- **Renomear depois de já ter usado o nome antigo** em outras células: reexecute o notebook inteiro do começo.

NO RELATÓRIO

Colunas **antes** e **depois** da renomeação, o código e a lista das **boas práticas** aplicadas.

Passo 7.4 — Identificação e tratamento dos tipos de dados

O que é

Verificar o **tipo** que o pandas atribuiu a cada coluna e corrigir os que não correspondem ao que a coluna realmente representa.

Por que fazer

O tipo define o que é possível fazer com a coluna. Não se calcula média de um texto; ordenar números armazenados como texto coloca “10” antes de “9”; datas só funcionam em gráficos de linha e cálculos de intervalo se forem datas de verdade. Além disso, convém distinguir o **tipo computacional** do **tipo estatístico**: o código postal 3067 é um número, mas é um **código** (categoria), e calcular a média de códigos postais não tem sentido.

Os principais tipos

dtype	Representa	Exemplo
int64	Números inteiros (sem ausentes)	rooms: 2, 3, 4
float64	Números decimais (aceita ausentes)	price: 1480000.0
Int64 (I maiúsculo)	Inteiros que aceitam ausentes (nullable)	postcode: 3067
str / object	Texto	suburb: Abbotsford
category	Poucas categorias que se repetem; economiza memória	type: h, u, t
bool	Verdadeiro/falso	True / False
datetime64	Datas e horas	2016-12-03

Como fazer

```
print(df.dtypes)                                # antes

df["postcode"] = df["postcode"].astype("Int64")
df["property_count"] = df["property_count"].astype("Int64")

for coluna in ["type", "method", "region_name", "council_area"]:
    df[coluna] = df[coluna].astype("category")

print(df.dtypes)                                # depois
```

Código	O que acontece
df.dtypes	Lista o tipo de cada coluna. É o ponto de partida do diagnóstico.
df["postcode"].astype("Int64")	Converte o código postal, que veio como decimal (3067.0), para inteiro. Usa Int64 (com I maiúsculo) para tolerar ausentes.
df["property_count"]...	Mesma lógica: quantidade de imóveis no bairro é uma contagem inteira.
for coluna in [...]	Repete o mesmo comando para várias colunas, sem copiar código.
.astype("category")	Marca colunas de texto com poucos valores distintos como categóricas . Ficam mais leves e explícitas.

O que você deve ver

SAÍDA ESPERADA

ANTES:		DEPOIS:	
suburb	str	suburb	str
rooms	int64	rooms	int64
type	str	type	category
price	float64	price	float64
postcode	float64	postcode	Int64
property_count	float64	property_count	Int64
...		...	

COMO DECIDIR NO SEU PROJETO

Perguntas para cada coluna da **sua** base:

- O tipo atual é compatível com o que a coluna representa? (valor monetário em texto, data em texto, código como número...)
- Os valores com símbolo (R\$, %, vírgula decimal) precisam ser limpos antes de virar número?
- Existem poucas categorias repetidas? Então talvez *category*.
- A coluna tem ausentes? Então inteiro precisa ser *Int64* (ou trate os ausentes antes).

Convertendo texto “sujo” em número

```
# "R$ 1.234,56" -> 1234.56
df["valor"] = (df["valor"].str.replace("R$", "", regex=False)
               .str.replace(".", "", regex=False)
               .str.replace(",", ".", regex=False)
               .astype(float))

# se houver textos impossíveis de converter, transforme-os em ausentes:
df["valor"] = pd.to_numeric(df["valor"], errors="coerce")
```

Erros comuns

- **“Cannot convert NA to integer”**: *astype(int)* não aceita ausentes. Use *Int64* ou trate os ausentes antes.
- **Converter e esquecer de reatribuir**: *df["x"].astype(...)* sozinho não altera a coluna.
- **Tratar como número o que é código** (CEP, código postal, CNPJ): não faz sentido somar, tirar média nem discretizar.

NO RELATÓRIO

Os tipos de dados **antes** e **depois**, o código das conversões e a justificativa de cada correção (por que o tipo original estava inadequado).

Passo 7.5 — Variáveis de data e hora

O que é

Identificar colunas que representam datas ou horários e convertê-las para o tipo **datetime**. Na base de exemplo, *date* vem como texto no formato *3/12/2016*.

Por que fazer

- Permite **ordenar** cronologicamente, **filtrar** por período e calcular **intervalos** entre datas.
- Permite extrair **componentes** (ano, mês, dia da semana) para criar novas variáveis e gráficos temporais.

Como fazer

```
df["date"] = pd.to_datetime(df["date"], dayfirst=True)

print("Tipo de 'date' após a conversão:", df["date"].dtype)
print(f"Período: {df['date'].min().date()} a {df['date'].max().date()}")
```

Código	O que acontece
<code>pd.to_datetime(...)</code>	Converte texto em datas reais (<i>datetime64</i>).
<code>dayfirst=True</code>	Informa que o primeiro número é o dia (formato dia/mês/ano). Sem isso, o pandas pode assumir mês/dia/ano.
<code>df['date'].min() / .max()</code>	Menor e maior data: mostram o período coberto pela base, informação exigida na seção 4 do relatório.

O que você deve ver

SAÍDA ESPERADA

Tipo de 'date' após a conversão: `datetime64[us]`
 Período: 2016-01-28 a 2017-09-23

ATENÇÃO

Dia/mês trocados não geram erro. “3/12/2016” pode ser 3 de dezembro ou 12 de março, e o pandas converte os dois sem reclamar. Para descobrir o formato, procure uma data em que o primeiro número seja maior que 12: na base de exemplo existem datas como 30/..., que só podem ser dia. Uma alternativa ainda mais segura é declarar o formato: `pd.to_datetime(df["date"], format="%d/%m/%Y")`.

O que fazer depois da conversão (opcional)

```
df["ano_venda"] = df["date"].dt.year      # 2016, 2017
df["mes_venda"] = df["date"].dt.month    # 1 a 12
df["dia_semana"] = df["date"].dt.day_name() # Monday, Tuesday...
```

O acessor `.dt` dá acesso aos componentes da data. Esses comandos são apenas ilustrativos: o notebook de exemplo não cria essas colunas, mas o seu projeto pode criá-las se forem úteis para as suas questões.

Se a sua base não tiver datas

Declare isso expressamente no relatório, com a frase pedida no template: **“A base de dados utilizada não possui variáveis de data e hora.”**

Erros comuns

- **Datas trocadas** (dia × mês) por esquecer o *dayfirst* ou o *format*.
- **NaT (Not a Time)**: valores que não puderam ser convertidos. Se usar *errors="coerce"*, confira `df["date"].isna().sum()`.
- Esquecer de **reatribuir** o resultado na coluna.

NO RELATÓRIO

Identificação da(s) coluna(s) de data, código da conversão, **período coberto** e a **finalidade** da variável no projeto (por exemplo, analisar vendas ao longo do tempo).

Passo 7.6 — Dados ausentes

O que é

Valores que **faltam** na base. No pandas aparecem como **NaN** (*Not a Number*) ou, em colunas de texto, como vazio. Tratá-los exige três decisões: **quantos** faltam, **por que** faltam e **o que fazer** a respeito.

Passo A — Medir

```
df.loc[df["building_area"] == 0, "building_area"] = np.nan # zero impossível vira ausente

ausentes = df.isna().sum() # quantidade por coluna
percentual = (df.isna().mean() * 100).round(2) # percentual por coluna
resumo = pd.DataFrame({"qtd_ausentes": ausentes, "pct_ausentes": percentual})
resumo[resumo["qtd_ausentes"] > 0]
```

Código	O que acontece
<code>df.loc[df["building_area"] == 0, "building_area"] = np.nan</code>	Ausentes disfarçados: uma área construída de 0 m ² é impossível, então é um dado faltante. Converter para NaN o torna visível ao <i>isna</i> .
<code>df.isna().sum()</code>	<i>isna()</i> marca com True cada célula vazia; <i>sum()</i> conta quantas há em cada coluna.
<code>df.isna().mean() * 100</code>	A média de True/False é a proporção de ausentes. Multiplicada por 100, vira percentual.
<code>pd.DataFrame({...})</code>	Junta contagem e percentual em uma tabela para exibir e colar no relatório.

SAÍDA ESPERADA

	qtd_ausentes	pct_ausentes
car	62	0.46
building_area	6467	47.62
year_built	5375	39.58
council_area	1369	10.08

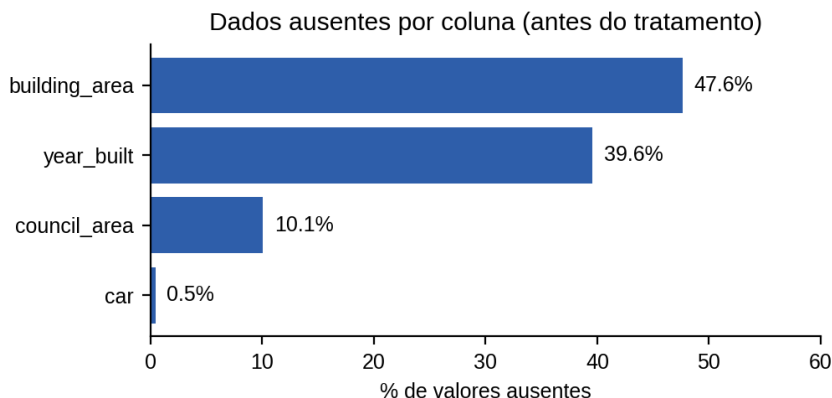


Figura 1 — Percentual de dados ausentes por coluna, antes do tratamento.

ATENÇÃO

Procure ausentes disfarçados. Nem todo dado faltante aparece como NaN. Fique atento a 0, -1, 999, "N/A", "?", "-" e textos vazios. Na base de exemplo, 17 linhas tinham *building_area* = 0. Já os 1.939 valores *landsize* = 0 são **legítimos** (1.629 são apartamentos/unidades, que não têm terreno próprio): nem todo zero é erro, é preciso entender o contexto.

Passo B — Escolher a técnica

Não existe técnica universal. A escolha depende do **tipo da variável**, do **percentual** de ausentes e do **papel** da variável no projeto.

Situação	Técnicas possíveis	Cuidados
Poucos ausentes (menos de 5%), numérica discreta ou categórica	Preencher com a moda (valor mais frequente), ou remover as linhas (<i>dropna</i>)	Remover só vale se o número de linhas perdidas for pequeno
Numérica contínua	Preencher com a mediana (global ou por grupo)	Prefira mediana à média quando houver valores extremos
Categórica	Categoria nova, como “Não informado”, ou a moda	Criar categoria preserva a informação de que o dado faltava
Muitos ausentes (acima de 30–40%)	Imputar por grupo com cautela, ou excluir a coluna ; considerar uma coluna-indicador de ausência	Os valores imputados são estimativas, não dados reais: documente como limitação

Média ou mediana? Na base de exemplo, a área construída tem mediana de 126 m², mas máximo de 44.515 m² (um valor absurdo). Um único valor assim puxa a média para cima; a mediana não se deixa enganar. Por isso a mediana é mais robusta.

Passo C — Tratar

Escolhas feitas para cada coluna do exemplo:

Coluna	Ausentes	Técnica	Justificativa
car	62 (0,46%)	Moda	Quase nada falta e a variável é discreta; a moda preserva a distribuição.
council_area	1.369 (10,08%)	Categoria “Não informado”	Sem forma confiável de deduzir o conselho; criar a categoria evita inventar informação.
building_area	6.467 (47,62%)	Mediana por <i>type</i> e <i>rooms</i>	Quase metade falta, então descartar linhas custaria demais. A área varia com tipo e nº de cômodos, daí a mediana do grupo.
year_built	5.375 (39,58%)	Mediana por <i>suburb</i>	Imóveis do mesmo bairro tendem a ter idades próximas.

```
# car: moda
moda_car = df["car"].mode(dropna=True)[0]
df["car"] = df["car"].fillna(moda_car)

# council_area: categoria nova (precisa ser adicionada antes, pois a coluna e 'category')
if "Não informado" not in df["council_area"].cat.categories:
    df["council_area"] = df["council_area"].cat.add_categories(["Não informado"])
df["council_area"] = df["council_area"].fillna("Não informado")

# building_area: mediana por tipo e numero de comodoss
df["building_area"] = df.groupby(["type", "rooms"])["building_area"].transform(
    lambda s: s.fillna(s.median())
)
df["building_area"] = df["building_area"].fillna(df["building_area"].median()) # residuais

# year_built: mediana por bairro
df["year_built"] = df.groupby("suburb")["year_built"].transform(
    lambda s: s.fillna(s.median())
)
df["year_built"] = df["year_built"].fillna(df["year_built"].median()) # residuais
```

Código	O que acontece
<code>df["car"].mode(dropna=True)[0]</code>	<i>mode()</i> devolve os valores mais frequentes; <i>[0]</i> pega o primeiro. Aqui, 2 vagas.
<code>.fillna(valor)</code>	Preenche apenas as células vazias com o valor indicado; as demais ficam intactas.

Código	O que acontece
<code>cat.add_categories(...)</code>	Colunas <i>category</i> só aceitam valores já cadastrados. É preciso cadastrar “Não informado” antes de usá-lo.
<code>df.groupby(["type", "rooms"])</code>	Separa as linhas em grupos (por exemplo, casas com 3 cômodos; unidades com 2 cômodos...).
<code>.transform(lambda s: ...)</code>	Aplica uma função a cada grupo e devolve um resultado com o mesmo tamanho da coluna original, de modo que cada linha recebe o valor do seu grupo.
<code>s.fillna(s.median())</code>	Dentro de cada grupo <i>s</i> , preenche os vazios com a mediana do próprio grupo .
<code>... .fillna(df[...].median()) # residuais</code>	Rede de segurança: se um grupo inteiro estiver vazio (sem mediana possível), usa a mediana geral.

Passo D — Conferir

SAÍDA ESPERADA

Dados ausentes DEPOIS do tratamento:
Nenhuma coluna com dados ausentes.

ATENÇÃO

Limitação a registrar. Cerca de 48% das áreas construídas e 40% dos anos de construção agora são **estimativas**, não medições reais. Resultados que dependem dessas colunas (como o preço por m²) devem ser interpretados com cautela e essa limitação deve aparecer na discussão do relatório final (Etapa II).

COMO DECIDIR NO SEU PROJETO

Para cada coluna com ausentes da **sua** base:

- Quantos faltam, em número e em %?
- Por que podem estar faltando (não se aplica, não foi coletado, erro de registro)?
- A variável é numérica ou categórica? Tem valores extremos?
- Existe um grupo natural (tipo, região, categoria) que permita preencher de forma mais realista que um valor global?
- Vale mais descartar a coluna ou as linhas? Justifique com números.

Erros comuns

- **TypeError ao preencher uma coluna *category* com um valor novo:** use *add_categories* antes.
- **Usar *inplace=True* em uma coluna** (`df["x"].fillna(0, inplace=True)`): em versões recentes do pandas isso pode não ter efeito. Prefira `df["x"] = df["x"].fillna(0)`.
- **Apagar linhas sem medir o custo.** Na base de exemplo, *dropna()* removeria 7.385 das 13.580 linhas (54%).
- **Esquecer de tratar os ausentes disfarçados** (zeros, “N/A”) antes de calcular médias e medianas.

NO RELATÓRIO

Tabela de ausentes **antes** (quantidade e %), a técnica escolhida **para cada coluna** com justificativa que considere o tipo da variável e o conjunto de dados, o código do tratamento e a tabela de ausentes **depois**.

Passo 7.7 — Tratamento e padronização dos valores

O que é

Procurar e corrigir **inconsistências** nos valores: grafias diferentes para a mesma coisa, espaços sobrando, letras maiúsculas e minúsculas misturadas, abreviações, erros de digitação e números impossíveis.

Por que fazer

Para o computador, “Brunswick”, “brunswick” e “Brunswick ” (com espaço no fim) são três categorias diferentes. Se isso acontecer, tabelas de frequência, agrupamentos e gráficos ficam errados sem nenhum aviso.

Como detectar

Problema	Como encontrar	Como corrigir
Espaços extras	<code>(serie != serie.str.strip()).sum()</code>	<code>.str.strip()</code> e <code>.str.replace(r"\s+", " ", regex=True)</code>
Maiúsculas/minúsculas	Comparar <code>nunique()</code> com <code>str.lower().nunique()</code>	<code>.str.lower()</code> , <code>.str.upper()</code> ou <code>.str.title()</code>
Abreviações e sinônimos	<code>value_counts()</code> e <code>unique()</code> para ver as variações	<code>.replace({"SP": "São Paulo"})</code>
Erros de digitação	Ler a lista de categorias; <code>describe()</code> para números	<code>.replace()</code> ou <code>.loc</code> com a correção
Números impossíveis	<code>describe()</code> , <code>sort_values()</code> , <code>boxplot</code>	<code>.loc</code> para corrigir, ou transformar em NaN
Linhas duplicadas	<code>df.duplicated().sum()</code>	<code>df.drop_duplicates()</code>

Como fazer

```
# 1) Espacos extras e grafias que so diferem na caixa
for coluna in ["suburb", "region_name"]:
    antes = df[coluna].astype(str)
    depois = antes.str.strip().str.replace(r"\s+", " ", regex=True)
    print(f"{coluna}: {(antes != depois).sum()} valores alterados (espacos extras).")
    print(f"{coluna}: {depois.nunique()} categorias;",
          f"{depois.str.lower().nunique()} ignorando caixa.")
    df[coluna] = depois if coluna == "suburb" else depois.astype("category")

# 2) Erro de digitacao: ano de construcao 1196 (certamente 1996)
erro = df["year_built"] < 1800 # mascara: anos impossiveis
df.loc[erro, "year_built"] = df.loc[erro, "year_built"] + 800

# 3) Linhas duplicadas (podem surgir ao excluir colunas identificadoras)
print("Linhas duplicadas:", df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)
```

Código	O que acontece
<code>antes.str.strip()</code>	Remove espaços no início e no fim do texto.
<code>.str.replace(r"\s+", " ", regex=True)</code>	Troca qualquer sequência de espaços por um único espaço (corrige “Santa Cruz” com dois espaços).
<code>depois.nunique()</code> vs <code>depois.str.lower().nunique()</code>	Se os dois números forem iguais, não há grafias que diferem só na caixa. Se o segundo for menor, há categorias duplicadas.
<code>erro = df["year_built"] < 1800</code>	Cria a máscara (True/False) das linhas com ano impossível (anterior a 1800).
<code>df.loc[erro, "year_built"] = ... + 800</code>	Nessas linhas, soma 800 ao ano (1196 vira 1996), corrigindo só a coluna <code>year_built</code> .
<code>df.drop_duplicates().reset_index(drop=True)</code>	Remove linhas inteiramente repetidas e reorganiza a numeração do índice.

O que você deve ver

SAÍDA ESPERADA

```
suburb: 0 valores alterados (espacos extras).
suburb: 314 categorias; 314 ignorando caixa.
region_name: 0 valores alterados (espacos extras).
region_name: 8 categorias; 8 ignorando caixa.

Valores minimo e maximo de year_built ANTES: 1196.0 - 2018.0
Valores minimo e maximo de year_built DEPOIS: 1830.0 - 2018.0
Linhas duplicadas: 1
Linhas apos remover duplicadas: 13579
```

ATENÇÃO

Padronizar não é aplicar tudo a todas as colunas. Aplicar `.str.title()` automaticamente em *suburb* mudaria 8 linhas do bairro *McKinnon* para *Mckinnon*, ou seja, **pioraria** um nome que estava certo. A regra é: **detecte, inspecione o que mudaria e só então corrija**. Se a verificação mostrar que não há problema, registrar isso no relatório também é uma resposta válida.

Corrigir ou não corrigir? O caso do ano 1196

O valor 1196 em *year_built* é obviamente um erro de digitação (Melbourne foi fundada em 1835). A hipótese mais plausível é que seja 1996, com um dígito errado, e por isso o exemplo soma 800. Mas isto é uma **suposição**. Quando não houver evidência clara, a alternativa mais honesta é transformar o valor em NaN e deixá-lo seguir o tratamento de ausentes. Seja qual for a escolha, **explique-a**.

ATENÇÃO

Valores extremos que o exemplo não corrige. A coluna *building_area* ainda contém 44.515 m² (a mediana, após o tratamento, é 130 m²) e, como consequência, o *price_per_m2* varia de 30 a 3,8 milhões de AUD/m². O exemplo mantém esses registros, mas um grupo criterioso deve investigá-los e decidir entre manter, limitar (*clip*), transformar em ausente ou remover, **documentando a decisão**. Ao resumir variáveis como essas, prefira a mediana.

NO RELATÓRIO

O que foi verificado, o que foi encontrado, as correções feitas (com o **antes e depois**) e a justificativa de cada alteração. Se não encontrou inconsistências em alguma coluna, escreva isso e mostre a verificação que fez.

Passo 7.8 — Criação de novas variáveis

O que é

Construir colunas novas a partir das existentes, para expressar conceitos que os dados brutos não trazem de forma direta. É pedido **pelo menos duas**: uma **numérica** e uma **categórica criada com .loc**.

7.8.1 — Variável numérica: price_per_m2

```
df["price_per_m2"] = (df["price"] / df["building_area"]).round(2)
df[["price", "building_area", "price_per_m2"]].head()
```

Código	O que acontece
<code>df["price"] / df["building_area"]</code>	Divide, linha a linha , o preço pela área. O pandas faz a operação em todas as linhas de uma vez (sem precisar de laço).
<code>.round(2)</code>	Arredonda para 2 casas decimais.
<code>df["price_per_m2"] = ...</code>	Atribuir a uma coluna que ainda não existe a cria .

SAÍDA ESPERADA

```
      price  building_area  price_per_m2
0  1480000.0           96.0       15416.67
1  1035000.0           79.0       13101.27
2  1465000.0          150.0        9766.67
```

Finalidade: comparar imóveis de tamanhos diferentes em pé de igualdade. Um apartamento de 1 milhão com 60 m² e uma casa de 1 milhão com 200 m² têm o mesmo preço, mas valores muito diferentes por m².

ATENÇÃO

Divisão por zero e valores extremos. Se *building_area* fosse 0, o resultado seria infinito; por isso os zeros foram tratados como ausentes no passo 7.6. Mesmo assim, o preço por m² tem mediana de 7.536 AUD/m² e valores extremos (mínimo 30, máximo 3,8 milhões): resuma com a **mediana**, não com a média.

Outras ideias de variáveis numéricas

- **Razões:** receita por funcionário, custo por unidade, vendas por loja.
- **Diferenças e idades:** 2026 – ano de fundação; dias entre pedido e entrega.
- **Somas e totais:** total de vendas = quantidade × preço unitário.
- **Componentes de data:** ano, mês, trimestre.

7.8.2 — Variável categórica com .loc: faixa_distancia

O `.loc[linhas, coluna]` seleciona **quais linhas** (por uma condição) e **qual coluna** você quer ler ou escrever. Aqui ele é usado para **escrever** um rótulo de texto nas linhas que cumprem cada regra.

```
df["faixa_distancia"] = ""
df.loc[df["distance"] <= 5, "faixa_distancia"] = "Até 5 km"
df.loc[(df["distance"] > 5) & (df["distance"] <= 10), "faixa_distancia"] = "5 a 10 km"
df.loc[(df["distance"] > 10) & (df["distance"] <= 20), "faixa_distancia"] = "10 a 20 km"
df.loc[df["distance"] > 20, "faixa_distancia"] = "Acima de 20 km"
df["faixa_distancia"] = df["faixa_distancia"].astype("category")
```

Código	O que acontece
<code>df["faixa_distancia"] = ""</code>	Cria a coluna nova, inicialmente vazia (texto vazio), que será preenchida por regra.
<code>df["distance"] <= 5</code>	Gera uma série de True/False , uma por linha: verdadeiro quando a distância é no máximo 5 km. Isso é a máscara de seleção.
<code>df.loc[máscara, "faixa_distancia"] = "Até 5 km"</code>	Nas linhas em que a máscara é verdadeira, grava o rótulo na coluna indicada.
<code>(a) & (b)</code>	O e lógico entre duas condições. Os parênteses são obrigatórios . Use <code>&</code> (e), <code> </code> (ou), <code>~</code> (não), nunca <i>and/or</i> .
<code>.astype("category")</code>	Marca a coluna como categórica, já que tem poucos valores repetidos.

Como conferir se as regras estão certas

```
df.groupby("faixa_distancia", observed=True)["distance"].agg(["min", "max", "count"])
(df["faixa_distancia"] == "").sum() # deve ser 0: ninguém ficou sem faixa
```

SAÍDA ESPERADA

	min	max	count
faixa_distancia			
10 a 20 km	10.1	20.0	5515
5 a 10 km	5.1	9.9	5096
Acima de 20 km	20.4	48.1	731
Até 5 km	0.0	5.0	2237

Cada faixa respeita os limites previstos, sem sobreposição e sem sobras. O total (13.579) fecha com o número de linhas da base.

Erros comuns

Sintoma	Causa	Correção
ValueError: truth value of a Series is ambiguous	Usou <i>and/or</i> ou faltaram parênteses	Usar <i>&/ </i> e parênteses em cada condição
Linhas com texto vazio na nova coluna	Faixas com buracos (ex.: > 5 e < 10 , esquecendo o 10)	Conferir limites e testar (<i>coluna == ""</i>). <i>sum()</i>
Rótulo errado em alguns registros	Faixas que se sobrepõem: a última regra vence	Garantir condições mutuamente exclusivas
SettingWithCopyWarning	Atribuir em um recorte: <i>df[df.x>5][“y”] = ...</i>	Usar <i>df.loc[condição, “y”] = ...</i>

COMO DECIDIR NO SEU PROJETO

Escolha variáveis novas que **respondam a uma necessidade do projeto**: ligadas às suas questões de investigação. Defina limites com base em um critério (regra de negócio, referência do setor ou distribuição dos dados) e explique-o. A variável categórica deve ser criada com *.loc*, e não com *np.where* ou *pd.cut*, pois é uma exigência do enunciado.

NO RELATÓRIO

Para **cada** variável: nome, descrição, finalidade no projeto, código e um trecho do dataframe com a coluna nova.

Passo 7.9 — Discretização de variáveis

O que é

Transformar uma variável **numérica contínua** em **faixas** (intervalos), obtendo uma variável categórica ordenada. Há duas técnicas pedidas no enunciado:

Técnica	Ideia	Função pandas
Equal Width (largura igual)	Divide o intervalo entre o mínimo e o máximo em faixas de mesmo tamanho	<i>pd.cut(coluna, bins=4)</i>
Equal Frequency (frequência igual)	Divide de modo que cada faixa tenha aproximadamente a mesma quantidade de registros (quantis)	<i>pd.qcut(coluna, q=4)</i>

Por que a escolha da técnica importa

Com **Equal Width**, o preço (de 85.000 a 9.000.000 AUD) seria dividido em quatro faixas de cerca de 2,2 milhões cada. Como a maioria dos imóveis custa menos de 2 milhões, quase tudo cai na primeira faixa: **12.926, 622, 28 e 3** imóveis. Com **Equal Frequency**, cada faixa fica com cerca de 3.394 imóveis. Veja a diferença:

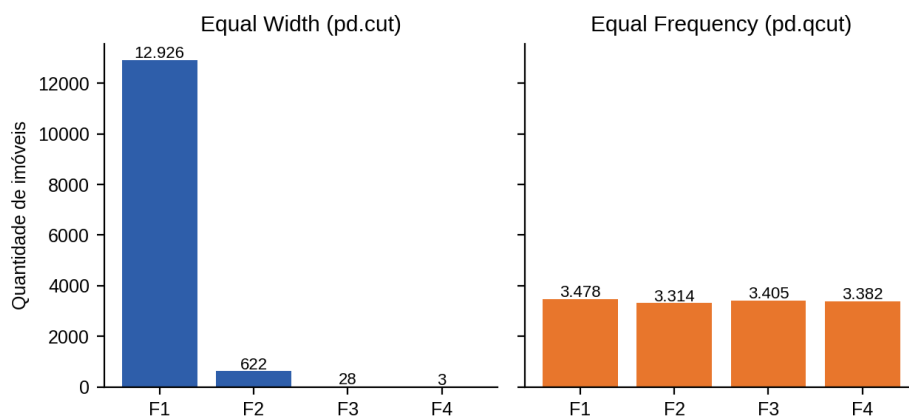


Figura 2 — Quantidade de imóveis por faixa de preço: Equal Width (esquerda) × Equal Frequency (direita).

Como fazer (Equal Frequency)

```
df["faixa_preco"] = pd.qcut(
    df["price"],
    q=4,
    labels=["Q1 - Mais baixo", "Q2 - Médio-baixo", "Q3 - Médio-alto", "Q4 - Mais alto"],
)

df["faixa_preco"].value_counts().sort_index()
```

Código	O que acontece
<code>pd.qcut(df['price'], q=4, ...)</code>	Divide a coluna em 4 quantis (quartis): cada faixa guarda ~25% dos registros.
<code>labels=[...]</code>	Nomes das faixas, em ordem crescente. Sem isso, o pandas usa intervalos numéricos como rótulo.
<code>value_counts().sort_index()</code>	Conta quantos imóveis há em cada faixa e ordena pelo rótulo.

O que você deve ver

Faixa	Preço mínimo (AUD)	Preço máximo (AUD)	Imóveis
Q1 - Mais baixo	85.000	650.000	3.478
Q2 - Médio-baixo	650.500	903.000	3.314
Q3 - Médio-alto	904.000	1.330.000	3.405
Q4 - Mais alto	1.330.500	9.000.000	3.382

Os tamanhos das faixas não são exatamente iguais (3.478, 3.314, 3.405, 3.382) porque há muitos imóveis com **preços idênticos** (por exemplo, 1.000.000), e todos os empatados precisam ficar na mesma faixa.

Como interpretar os grupos

Descreva, com os limites, o que cada faixa representa. Exemplo: “A faixa Q1 reúne os imóveis mais acessíveis, com preços de até 650 mil AUD; a faixa Q4 reúne os 25% mais caros, a partir de 1,33 milhão. A amplitude dentro de Q4 (de 1,33 a 9 milhões) é bem maior que nas demais, mostrando a cauda de imóveis de luxo.”

COMO DECIDIR NO SEU PROJETO

Para escolher a variável e a técnica na **sua** base:

- A variável é **contínua** (idade, valor, tempo, peso) e importante para o projeto?
- A distribuição é **assimétrica**, com poucos valores muito altos? Então Equal Frequency. É razoavelmente uniforme, ou há faixas naturais (idade em décadas)? Então Equal Width.
- Quantas faixas? Entre 3 e 5 costuma ser suficiente. Justifique.
- Que nomes de faixa tornam a interpretação mais clara para quem lê o relatório?

Erros comuns

- **ValueError: Bin edges must be unique** ao usar `qcut`: muitos valores repetidos geram limites iguais. Reduza `q` ou use `duplicates="drop"`.
- **Quantidade de labels diferente de q**: precisa haver exatamente um rótulo por faixa.
- **Confundir discretização com a variável criada com .loc**: ambas geram categorias, mas aqui a divisão é feita por uma **técnica estatística** (largura ou frequência), não por regras escolhidas à mão.
- **Discretizar uma variável que não é contínua** (código, identificador).

NO RELATÓRIO

Variável escolhida, técnica utilizada, **justificativa** da técnica, código, resultado (contagem por faixa) e **interpretação** dos grupos formados.

Passo 7.10 — Variáveis dummy

O que é

Transformar uma variável categórica **nominal** (sem ordem natural) em várias colunas **binárias** (0 ou 1), uma para cada categoria. Também se chama *one-hot encoding* (codificação “um-quente”).

Por que fazer

Modelos estatísticos e de machine learning trabalham com números. Se você trocar “casa, unidade, sobrado” por 1, 2 e 3, estará dizendo ao modelo que sobrado é “mais” que unidade e que existe uma distância entre elas, e isso é falso. As dummies resolvem: cada categoria vira uma coluna que só diz **sim (1)** ou **não (0)**.

type	type_h	type_t	type_u
h (casa)	1	0	0
t (sobrado)	0	1	0
u (unidade)	0	0	1

Como fazer

```
dummies_type = pd.get_dummies(df["type"], prefix="type", dtype=int)
df = pd.concat([df, dummies_type], axis=1)

print("Novas colunas criadas:", dummies_type.columns.tolist())
df[["type"] + dummies_type.columns.tolist()].head()
```

Código	O que acontece
<code>pd.get_dummies(df["type"], ...)</code>	Cria uma coluna binária para cada categoria da coluna <i>type</i> .
<code>prefix="type"</code>	Prefixo dos nomes novos: <i>type_h</i> , <i>type_t</i> , <i>type_u</i> .
<code>dtype=int</code>	Pede colunas com 0 e 1 . Sem isso, versões recentes do pandas devolvem <i>True/False</i> .
<code>pd.concat([df, dummies_type], axis=1)</code>	Cola as colunas novas ao lado (<i>axis=1</i> = colunas) do DataFrame original.

SAÍDA ESPERADA

```
Novas colunas criadas: ['type_h', 'type_t', 'type_u']
   type  type_h  type_t  type_u
0     h        1        0        0
1     h        1        0        0
```

Conceitos que você deve dominar

- **Nominal × ordinal:** dummies são para variáveis **nominais**. Uma variável **ordinal**, como *faixa_preco* ($Q1 < Q2 < Q3 < Q4$), tem ordem e deve ser codificada preservando-a (1, 2, 3, 4), não em dummies.
- **k categorias → k colunas:** a última coluna é dedutível das outras (se não é casa nem sobrado, é unidade). Em regressão, costuma-se usar `drop_first=True` para evitar redundância (a chamada *armadilha da variável dummy*). Qualquer das opções é aceitável, desde que você **explique a escolha**.
- **Cuidado com muitas categorias:** a coluna *suburb* tem 314 valores; criaria 314 colunas. Para dummies, escolha uma variável com poucas categorias (aqui, *type* tem 3; *region_name* tem 8; *method* tem 5).

Erros comuns

- **Esquecer o `concat`:** as dummies existem, mas não entram no DataFrame.
- **Executar a célula duas vezes:** as colunas se duplicam (`type_h` repetida). Reinicie o kernel.
- **Ausentes:** `get_dummies` ignora NaN, deixando a linha com 0 em todas as colunas, a menos que use `dummy_na=True`.
- **Aplicar em variável ordinal ou de alta cardinalidade.**

NO RELATÓRIO

Variável escolhida e **justificativa**, resultado **antes** (coluna original) e **depois** (dummies), código e **explicação das novas variáveis** (o que significa 1 e 0 em cada uma).

Passo 7.11 — Estrutura final do dataframe

O que é

Um **balanço** do que mudou: quantas linhas e colunas a base tem agora, com que nomes e tipos, em comparação com a estrutura original.

Como fazer

```
print(f"Linhas: original = 13580 | final = {df.shape[0]}")
print(f"Colunas: original = 21 | final = {df.shape[1]}")
df.dtypes
```

SAÍDA ESPERADA

```
Linhas: original = 13580 | final = 13579
Colunas: original = 21 | final = 24
```

A conta das colunas fecha: 21 originais – 3 excluídas + 6 novas (`price_per_m2`, `faixa_distancia`, `faixa_preco`, `type_h`, `type_t`, `type_u`) = 24. As linhas passaram de 13.580 para 13.579 por causa da única linha duplicada removida no passo 7.7.

Tabela da estrutura final (24 colunas)

Coluna	Tipo no notebook	Origem	Observação
suburb	texto	original	—
rooms	inteiro	original	—
type	categoria	original (tratada)	→ category
price	decimal	original	—
method	categoria	original (tratada)	→ category
date	datetime	original (tratada)	texto → datetime
distance	decimal	original	—
postcode	Int64	original (tratada)	float → Int64
bathroom	decimal	original	—
car	decimal	original (tratada)	ausentes: moda
landsize	decimal	original	—
building_area	decimal	original (tratada)	zeros e ausentes: mediana por grupo
year_built	decimal	original (tratada)	ausentes: mediana por bairro; 1196 → 1996
council_area	categoria	original (tratada)	→ category; ausentes = “Não informado”
latitude	decimal	original	—

Coluna	Tipo no notebook	Origem	Observação
longitude	decimal	original	—
region_name	categoria	original (tratada)	→ category
property_count	Int64	original (tratada)	float → Int64
price_per_m2	decimal	nova	preço ÷ área construída
faixa_distancia	categoria	nova	criada com .loc
faixa_preco	categoria	nova	qcut, 4 faixas
type_h	inteiro (0/1)	nova	dummy de type
type_t	inteiro (0/1)	nova	dummy de type
type_u	inteiro (0/1)	nova	dummy de type

Checagens finais antes de exportar

- `df.isna().sum().sum()` deve ser **0** nas colunas que você tratou.
- `df.duplicated().sum()` deve ser **0**.
- `df.describe()`: confira mínimos e máximos das colunas numéricas.
- `df.dtypes`: confira se cada coluna tem o tipo previsto.

NO RELATÓRIO

Quantidade de linhas e colunas, nomes e tipos, **comparação com a estrutura original** e a explicação das **principais alterações realizadas** (colunas excluídas, tipos corrigidos, ausentes tratados, variáveis criadas, linhas removidas).

Passo 8 — Tabelas de frequência

O que é

Uma tabela que mostra **quantas vezes** cada categoria de uma variável aparece (frequência **absoluta**) e que parcela do total isso representa (frequência **relativa**, em %).

Como fazer

```
tab_type = df["type"].value_counts().to_frame("qtd")
tab_type["pct"] = (df["type"].value_counts(normalize=True) * 100).round(1)
tab_type
```

Código	O que acontece
<code>df["type"].value_counts()</code>	Conta quantas linhas existem para cada categoria, da mais para a menos frequente.
<code>.to_frame("qtd")</code>	Transforma o resultado em uma tabela (DataFrame) com a coluna <i>qtd</i> .
<code>value_counts(normalize=True)</code>	Em vez de contagens, devolve proporções (de 0 a 1).
<code>* 100round(1)</code>	Converte em percentual e arredonda em 1 casa decimal.

Resultados para o exemplo

Variável: type (tipo do imóvel)

type	qtd	pct
h	9.449	69,6%
u	3.017	22,2%
t	1.113	8,2%

Variável: region_name (região)

region_name	qtd	pct
Southern Metropolitan	4.695	34,6%
Northern Metropolitan	3.889	28,6%
Western Metropolitan	2.948	21,7%
Eastern Metropolitan	1.471	10,8%
South-Eastern Metropolitan	450	3,3%
Eastern Victoria	53	0,4%
Northern Victoria	41	0,3%
Western Victoria	32	0,2%

Variável: faixa_distancia (criada no passo 7.8.2)

faixa_distancia	qtd	pct
Até 5 km	2.237	16,5%
5 a 10 km	5.096	37,5%
10 a 20 km	5.515	40,6%
Acima de 20 km	731	5,4%

Como interpretar

Uma boa interpretação **descreve o padrão, cita os números e levanta uma explicação ou ressalva**:

- “Casas representam 69,6% das vendas (9.449 imóveis), contra 22,2% de unidades e 8,2% de sobrados. O mercado analisado é dominado por casas.”
- “Três regiões (Southern, Northern e Western Metropolitan) concentram 84,9% das vendas. As três regiões ‘Victoria’ somam apenas 126 imóveis (0,9%), o que torna qualquer média calculada para elas pouco confiável.”
- “78,1% dos imóveis estão entre 5 e 20 km do centro.”

COMO DECIDIR NO SEU PROJETO

Quais variáveis da **sua** base merecem tabela de frequência?

- As categóricas **relevantes** para suas questões de investigação (a base final, incluindo as criadas).
- Não faça tabela de identificadores (nome, código): teriam uma linha para cada registro.
- Se a variável tem dezenas de categorias, mostre as 10 mais frequentes (`.head(10)`) e agrupe o restante em “Outros”.

Erros comuns

- **value_counts()** ignora os ausentes. Para vê-los, use `dropna=False`.
- Os percentuais arredondados podem somar **99,9% ou 100,1%**. É normal, mas confira o total.
- **Só descrever a tabela** (“a categoria A tem 500”) sem dizer o que isso significa para o projeto.

NO RELATÓRIO

Uma tabela para **cada variável categórica relevante** da base final, o código e a **interpretação** dos principais padrões.

Passo 9 — Visualização dos dados

Como escolher o gráfico

Escolha pelo **tipo de dado** e pela **pergunta** que o gráfico responde:

Objetivo	Gráfico adequado	Exemplo neste projeto
Ver a distribuição de uma variável numérica	Histograma; boxplot	Preço dos imóveis
Comparar uma medida entre categorias	Barras/columnas; boxplot por categoria	Preço médio por região; preço por tipo
Ver a relação entre duas variáveis numéricas	Dispersão (scatter)	Distância × preço
Mostrar a evolução no tempo	Linha	Preço mediano por mês de venda
Mostrar partes de um todo (poucas categorias, até 5)	Setores (pizza) ou barras empilhadas	Participação por tipo

Regras para gráficos que se leem bem

- **Título e eixos com nome e unidade** (por exemplo, “Preço (milhões de AUD)”).
- **Barras sempre começam em zero.** Cortar o eixo exagera diferenças.
- **Ordene as categorias** de forma lógica (maior para menor, ou ordem natural das faixas).
- **Cores com propósito:** uma cor para uma medida; cores diferentes só para categorias diferentes, e sempre as mesmas em todos os gráficos.
- **Sem 3D nem decoração.** Pizza só com poucas categorias; rótulos legíveis.
- **Muitos pontos?** Em dispersões, use amostra e transparência (*alpha*) para evitar sobreposição.

Configuração comum (célula do notebook)

```
COR_PRINCIPAL = "#2E5EAA"
PALETA_CATEGORICA = ["#2E5EAA", "#E8762C", "#3C9F6B", "#A04CB0", "#C94C4C", "#5B6B7A"]
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False
```

Define as cores uma única vez, para reaproveitá-las em todos os gráficos, e remove as bordas superior e direita (visual mais limpo).

Gráfico 1 — Histograma do preço

Variáveis: price. **Por que histograma:** mostra a forma da distribuição de uma variável numérica contínua: onde os valores se concentram, se é simétrica ou tem cauda, se há valores isolados.

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(df["price"] / 1_000_000, bins=40, color=COR_PRINCIPAL, edgecolor="white")
ax.set_title("Distribuição do preço de venda dos imóveis")
ax.set_xlabel("Preço (milhões de AUD)")
ax.set_ylabel("Quantidade de imóveis")
plt.tight_layout()
plt.show()
```

Código	O que acontece
<code>plt.subplots(figsize=(8, 5))</code>	Cria a figura (<i>fig</i>) e uma área de desenho (<i>ax</i>), de 8 × 5 polegadas.
<code>ax.hist(..., bins=40)</code>	Desenha o histograma com 40 intervalos. Dividir por 1.000.000 expressa o preço em milhões.
<code>set_title</code> / <code>set_xlabel</code> / <code>set_ylabel</code>	Título e nomes dos eixos.
<code>plt.tight_layout()</code> / <code>plt.show()</code>	Ajusta margens para nada ser cortado e exibe o gráfico.

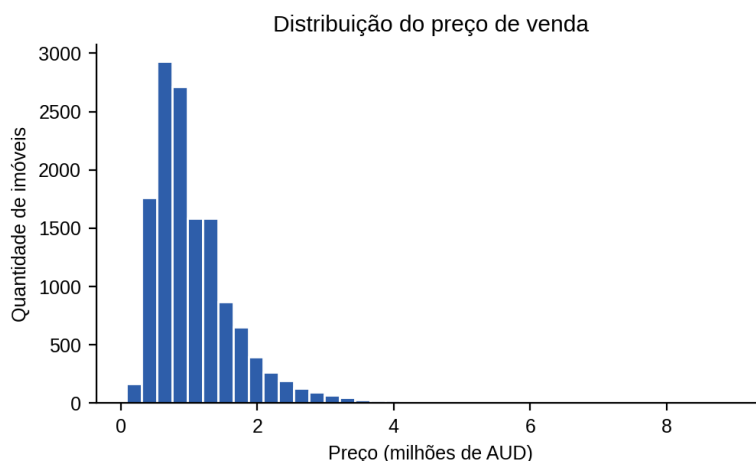


Figura 3 — Distribuição do preço de venda.

Como ler: cada barra conta quantos imóveis caem em uma faixa de preço. **Interpretação:** a distribuição é assimétrica à direita. 70,7% dos imóveis custam entre 500 mil e 1,5 milhão de AUD; a média (1.075.710) é maior que a mediana (903.000) porque uma cauda de imóveis caros puxa a média para cima. Isso justificou usar Equal Frequency no passo 7.9.

Gráfico 2 — Boxplot do preço por tipo de imóvel

Variáveis: price, type. **Por que boxplot:** compara a distribuição (mediana, dispersão e valores extremos) entre grupos. Responde à Questão 3.

```
ordem_tipos = ["h", "t", "u"]
dados_boxplot = [df.loc[df["type"] == t, "price"] / 1_000_000 for t in ordem_tipos]

fig, ax = plt.subplots(figsize=(8, 5))
caixas = ax.boxplot(dados_boxplot, patch_artist=True)
ax.set_xticks([1, 2, 3])
ax.set_xticklabels(["Casa (h)", "Sobrado (t)", "Unidade (u)"])
for patch, cor in zip(caixas["boxes"], PALETA_CATEGORICA):
    patch.set_facecolor(cor)
    patch.set_alpha(0.75)
ax.set_title("Preço de venda por tipo de imóvel")
ax.set_ylabel("Preço (milhões de AUD)")
plt.show()
```

Código	O que acontece
<code>[df.loc[df['type']==t, 'price'] ... for t in ...]</code>	<i>List comprehension:</i> cria uma lista com os preços de cada tipo, na ordem definida.
<code>ax.boxplot(dados, patch_artist=True)</code>	Desenha uma caixa para cada lista. <i>patch_artist=True</i> permite pintar as caixas.
<code>set_xticks / set_xticklabels</code>	Troca os rótulos 1, 2, 3 do eixo por nomes legíveis.
<code>for patch, cor in zip(...)</code>	Pinta cada caixa com uma cor da paleta.

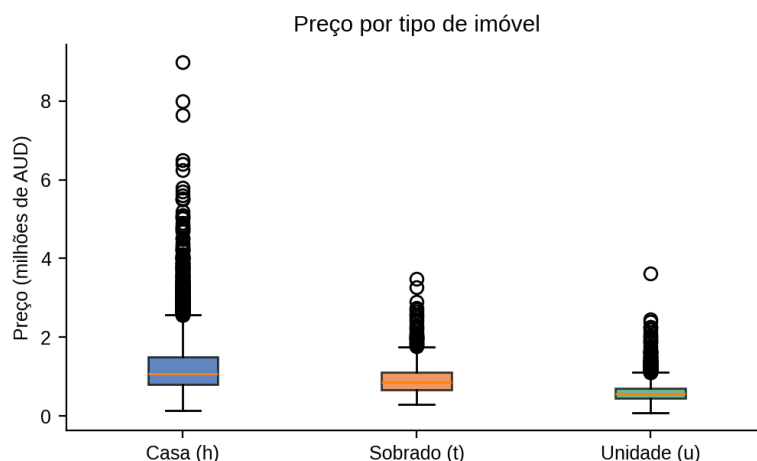


Figura 4 — Preço por tipo de imóvel.

Como ler: a caixa vai do 1º ao 3º quartil (50% centrais dos dados); a linha dentro é a **mediana**; as hastes se estendem até cerca de 1,5 vez a amplitude da caixa; os pontos fora são **valores atípicos (outliers)**.

Interpretação: a mediana das casas (1.080.000 AUD) é a maior, seguida pelos sobrados (847.500) e pelas unidades (560.000). As casas também têm maior dispersão e mais valores extremos.

Gráfico 3 — Barras: preço médio por região

Variáveis: price, region_name. **Por que barras:** comparar uma medida (média) entre categorias. Responde à Questão 1.

```
preco_medio_regiao = (df.groupby("region_name", observed=True)["price"].mean() / 1_000_000
                      ).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(preco_medio_regiao.index, preco_medio_regiao.values, color=COR_PRINCIPAL)
ax.set_title("Preço médio de venda por região")
ax.set_ylabel("Preço médio (milhões de AUD)")
plt.xticks(rotation=40, ha="right")
plt.show()
```

Código	O que acontece
<code>df.groupby("region_name")["price"].mean()</code>	Agrupar por região e calcular o preço médio de cada uma.
<code>sort_values(ascending=False)</code>	Ordenar da maior para a menor média, para leitura mais fácil.
<code>ax.bar(x, y)</code>	Desenhar uma barra por região.
<code>plt.xticks(rotation=40, ha="right")</code>	Inclinar os nomes no eixo X para não se sobreporem.

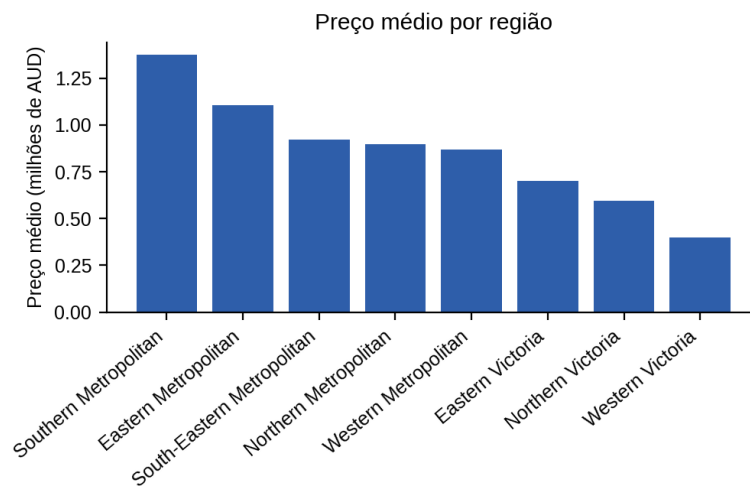


Figura 5 — Preço médio por região.

Interpretação: a região *Southern Metropolitan* tem o maior preço médio (1.372.963 AUD), seguida por *Eastern Metropolitan* (1.104.080). As regiões “Victoria”, mais afastadas, têm os menores valores (de 397.523 a 699.981).

Ressalva: essas três regiões somam apenas 126 imóveis, então a média delas é pouco confiável.

Gráfico 4 — Dispersão: distância × preço

Variáveis: distance, price. **Por que dispersão:** é o gráfico adequado para investigar a relação entre duas variáveis numéricas contínuas. Responde à Questão 2.

```
amostra = df.sample(n=2000, random_state=42)

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(amostra["distance"], amostra["price"] / 1_000_000,
           color=COR_PRINCIPAL, alpha=0.4, s=18)
ax.set_title("Distância ao centro x Preço de venda")
ax.set_xlabel("Distância ao centro (km)")
ax.set_ylabel("Preço (milhões de AUD)")
plt.show()
```

Código	O que acontece
<code>df.sample(n=2000, random_state=42)</code>	Sorteia 2.000 linhas. <i>random_state</i> fixa o sorteio para que o resultado seja reprodutível .
<code>alpha=0.4, s=18</code>	Transparência e tamanho dos pontos: onde há muitos pontos sobrepostos a cor fica mais forte.

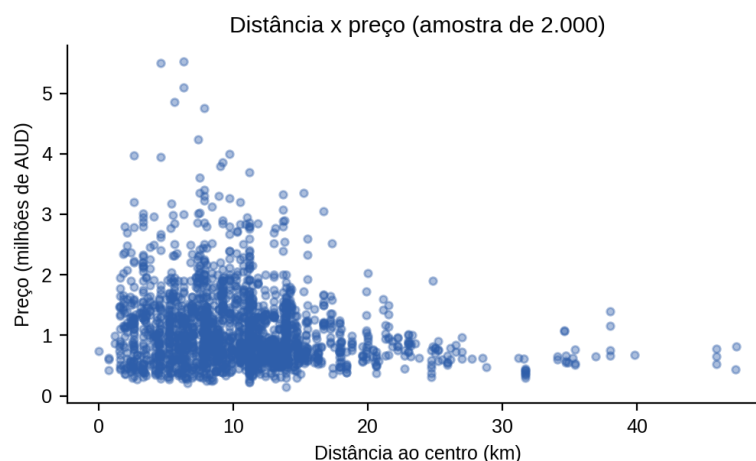


Figura 6 — Distância ao centro × preço (amostra de 2.000 imóveis).

Interpretação: a nuvem desce da esquerda para a direita, indicando que imóveis mais distantes tendem a custar menos, mas a relação é **fraca** (coeficiente de correlação de -0,16): há imóveis caros também em distâncias maiores. Em termos de mediana, o preço cai de 1.000.000 AUD (até 5 km) para 690.000 (acima de 20 km). Outros fatores (região, tipo, tamanho) também pesam. **Atenção:** correlação não é causalidade.

Gráfico 5 — Barras: imóveis por faixa de distância

Variáveis: faixa_distancia. **Por que barras:** contar registros por categoria; complementa o gráfico 4, mostrando onde está o volume de vendas.

```
contagem_faixa = df["faixa_distancia"].value_counts().reindex(
    ["Até 5 km", "5 a 10 km", "10 a 20 km", "Acima de 20 km"]
)
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(contagem_faixa.index, contagem_faixa.values, color=PALETA_CATEGORICA[1])
```

`reindex([...])` reorganiza as linhas na ordem lógica das faixas (e não na ordem de frequência).

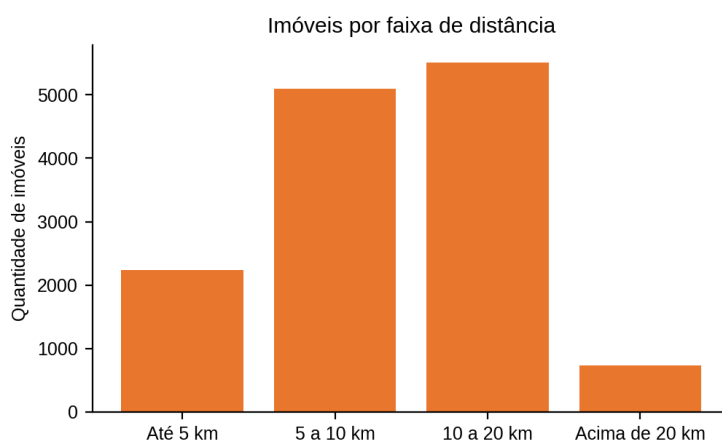


Figura 7 — Quantidade de imóveis por faixa de distância ao centro.

Interpretação: 78,1% das vendas se concentram entre 5 e 20 km do centro, e só 5,4% acima de 20 km.

COMO DECIDIR NO SEU PROJETO

Para cada gráfico da **sua** base: **(1)** qual pergunta ele responde? **(2)** que tipo de variável tenho? **(3)** o gráfico escolhido é o mais indicado para esse tipo? **(4)** consigo explicar o resultado com números?

Erros comuns

- **MatplotlibDeprecationWarning** ao usar *labels* em *boxplot* (parâmetro renomeado em versões novas): use *set_xticklabels*, como no exemplo.
- **Esquecer `plt.show()`** ou deixar dois gráficos misturados na mesma figura.
- **Nomes do eixo X sobrepostos:** use *plt.xticks(rotation=...)*.
- **Gráfico sem interpretação:** o relatório pede **variáveis, justificativa do tipo de gráfico e interpretação**.
- **Mostrar tudo que é possível** em vez do que é relevante. Poucos gráficos bem explicados valem mais.

NO RELATÓRIO

Para **cada gráfico**: as variáveis utilizadas, o **tipo de gráfico** e a **justificativa** da escolha, o código, o gráfico e a **interpretação** do resultado.

Passo 10 — Exportação da base preparada

O que é

Salvar o DataFrame final em um arquivo CSV, que será usado na Etapa II e entregue na pasta do Google Drive.

Como fazer

```
NUMERO_DO_GRUPO = "X"          # <-- substituir pelo número do seu grupo

nome_arquivo = f"base_projeto_grupo{NUMERO_DO_GRUPO}.csv"
df.to_csv(nome_arquivo, index=False)

print(f"Base exportada com sucesso: {nome_arquivo}")
print(f"Linhas: {df.shape[0]} | Colunas: {df.shape[1]}")
```

Código	O que acontece
NUMERO_DO_GRUPO = "X"	Guarda o número do grupo em uma variável. Troque o X antes de entregar (por exemplo, "7").
f"base_projeto_grupo{...}.csv"	Monta o nome obrigatório do arquivo: <i>base_projeto_grupo7.csv</i> , por exemplo.
df.to_csv(nome_arquivo, index=False)	Grava o CSV. index=False impede que a numeração das linhas (0, 1, 2...) seja gravada como uma coluna extra.

SAÍDA ESPERADA

```
Base exportada com sucesso: base_projeto_grupoX.csv
Linhas: 13579 | Colunas: 24
```

ATENÇÃO

Por que *index=False* é obrigatório? Sem ele, o arquivo ganha uma coluna sem nome. Ao reler, ela aparece como *Unnamed: 0*, poluindo a base. O enunciado exige CSV **sem índice**.

Conferindo se o arquivo ficou certo

```
teste = pd.read_csv(nome_arquivo)
print(teste.shape == df.shape)      # deve ser True
teste.head()
```

ATENÇÃO

O CSV não guarda tipos de dados. Ao reler o arquivo na Etapa II, datas voltam como texto e categorias como texto. Reconverta o que for necessário, por exemplo `pd.read_csv("base_projeto_grupoX.csv", parse_dates=["date"])`.

Onde o arquivo foi salvo?

- **Jupyter local:** na mesma pasta do notebook.
- **Google Colab:** em um ambiente **temporário**, que é apagado ao fechar a sessão. Baixe o arquivo logo: `from google.colab import files; files.download(nome_arquivo)`. Depois envie-o à pasta PROJETO_GRUPO_X do Google Drive.

Erros comuns

- **Esquecer de trocar o X** pelo número do grupo (o arquivo sai como *base_projeto_grupoX.csv*).
- **Esquecer `index=False`**.
- **Não encontrar o arquivo** depois de salvar (ambiente temporário do Colab).
- **Abrir no Excel e acentos quebrarem:** grave com `encoding="utf-8-sig"`.

NO RELATÓRIO

O código Python da exportação e a confirmação do nome do arquivo e do tamanho (linhas e colunas).

Montando e entregando a Parte I

Do notebook para o relatório

As seções do notebook seguem a mesma numeração do **Template do Relatório — Parte I**. Em cada seção, o relatório deve conter **código, um trecho do resultado, uma breve explicação e a justificativa**, quando aplicável.

Template do relatório	Onde está no notebook	Neste guia
1. Formação do grupo	Seção 1	—
2. Caracterização da empresa	Seção 2	—
3. Definição do projeto	Seção 3	—
4. Obtenção das bases de dados	Seção 4	Passo 7.1
5. Arquivos do projeto	Seção 5	Passo 10
6. Dicionário dos dados	Seção 6	Passos 7.1 e 7.4
7. Preparação e transformação (7.1 a 7.11)	Seção 7	Passos 7.1 a 7.11
8. Tabelas de frequência	Seção 8	Passo 8
9. Visualização dos dados	Seção 9	Passo 9
10. Exportação da base preparada	Seção 10	Passo 10
11. Conferência final	Seção 11	Checklist abaixo

Regras de entrega (resumo)

- Entrega pelo **SAVA**, em **um único PDF**, até **21/10/2026**.
- A entrega é **individual**: todos os integrantes sobem o arquivo, mesmo sendo o mesmo conteúdo. **Quem não entregar no prazo fica com nota zero.**
- O PDF deve ter **nome completo e matrícula de todos os integrantes**.
- **Textos obrigatórios no relatório**: comentários, justificativas e interpretações devem constar no relatório em PDF — **item de texto sem comentário é considerado errado (0%)**. No notebook (.ipynb) o preenchimento desses textos é **opcional**.
- Anexe ao PDF a **Carta de Autorização assinada** pela empresa (ou a **Declaração de Uso de Dados Públicos**). Sem ela, a atividade pode ser desconsiderada.
- Crie a pasta **PROJETO_GRUPO_X** no Google Drive, compartilhada com o professor Domingos Napolitano como editora, com a documentação, a base original, a base preparada e o código Python.

Consulte sempre o documento oficial **Instruções de Entrega — Parte I** para as regras completas.

Conferência final

☐ Utilizou dados reais de uma empresa ativa (ou dados públicos, com declaração)	☐ Realizou discretização, quando aplicável
☐ Possui autorização da empresa (ou declaração de dados públicos)	☐ Criou variáveis dummy
☐ Apresentou a origem dos dados	☐ Apresentou a estrutura final da base
☐ Apresentou o dicionário dos dados	☐ Criou tabelas de frequência
☐ Carregou a base utilizando Pandas	☐ Criou visualizações adequadas
☐ Verificou a quantidade de linhas e colunas	☐ Salvou a base final em CSV, sem índice, com o nome correto
☐ Avaliou e tratou os tipos de dados	☐ Disponibilizou todos os arquivos na pasta do Google Drive, compartilhada com o professor Domingos Napolitano
☐ Avaliou os dados ausentes	☐ Anexou a documentação de autorização da empresa
☐ Corrigiu inconsistências, quando existentes	☐ Nome e matrícula de todos os integrantes no arquivo
☐ Criou uma nova variável numérica	☐ Arquivo em PDF enviado pelo SAVA por CADA integrante, dentro do prazo
☐ Criou uma nova variável categórica utilizando .loc	

Guia rápido de erros e mensagens

Quando o Python mostrar uma mensagem vermelha, **leia a última linha**: ela diz o tipo do erro e o motivo. Os casos mais frequentes neste projeto:

Mensagem	O que significa	O que fazer
KeyError: 'coluna'	Coluna não existe (nome errado, já renomeada ou excluída)	Conferir <i>df.columns</i> ; reiniciar o notebook e rodar na ordem
FileNotFoundError	Arquivo não encontrado	Verificar o caminho; no Colab, enviar o arquivo antes
ParserError / uma coluna só	Separador errado	Usar <i>sep=";"</i> ou o separador correto
UnicodeDecodeError	Codificação de caracteres diferente	<i>encoding="latin-1"</i>
ValueError: truth value of a Series is ambiguous	Condição composta com <i>and/or</i> ou sem parênteses	Usar <i>&</i> , <i> </i> e parênteses
TypeError: Cannot setitem on a Categorical with a new category	Valor novo em coluna <i>category</i>	<i>cat.add_categories([...])</i> antes
IntCastingNaNError / Cannot convert NA to integer	Converter para <i>int</i> uma coluna com ausentes	Usar <i>Int64</i> ou tratar ausentes antes
SettingWithCopyWarning	Atribuição em um recorte do DataFrame	Usar <i>df.loc[cond, coluna] = valor</i>
ValueError: Bin edges must be unique	<i>qcut</i> com muitos valores repetidos	Reduzir <i>q</i> ou <i>duplicates="drop"</i>
NaT na coluna de data	Texto não convertido em data	Conferir formato e <i>dayfirst/format</i>
Coluna "Unnamed: 0" ao ler o CSV	O arquivo foi salvo com índice	Salvar com <i>index=False</i>
Nada mudou depois do comando	Resultado não foi reatribuído (<i>df = ...</i>)	Reatribuir: <i>drop</i> , <i>rename</i> , <i>astype</i> não alteram sozinhos

Glossário

Termo	Significado
DataFrame	Tabela de dados do pandas: linhas (registros) × colunas (variáveis).
Série (Series)	Uma única coluna de um DataFrame.
dtype	Tipo de dado de uma coluna (int64, float64, str, category, datetime64...).
NaN / NaT	Valor ausente (numérico) / data ausente.
Imputação	Preencher valores ausentes com uma estimativa (moda, mediana, valor de grupo).
Moda, mediana, média	Valor mais frequente; valor central; soma dividida pela quantidade. A mediana resiste a valores extremos.
Outlier (valor atípico)	Valor muito distante da maioria; pode ser erro ou um caso real raro.
Quantil / quartil	Pontos que dividem os dados ordenados em partes iguais (quartis: quatro partes de 25%).
Variável nominal / ordinal	Categoria sem ordem (tipo de imóvel) / categoria com ordem (faixa de preço).
Discreta / contínua	Numérica que só assume valores inteiros contáveis (nº de quartos) / que assume qualquer valor em um intervalo (preço).
Discretização	Transformar variável contínua em faixas.
Dummy (one-hot)	Coluna binária (0/1) que indica a presença de uma categoria.
snake_case	Convenção de nomes em minúsculas com palavras separadas por _.
Frequência absoluta / relativa	Contagem de ocorrências / proporção (ou %) do total.
Máscara booleana	Série de True/False usada para selecionar linhas (<code>df['x'] > 5</code>).
Cardinalidade	Quantidade de valores distintos de uma coluna.

Exercícios de fixação

Responda sem consultar o guia e confira as respostas na página seguinte.

1. Um CSV exportado do Excel em português tem colunas separadas por ;, números como 1234,56 e acentos que aparecem como “Ã£”. Como você escreveria o `read_csv`?
2. Você escreveu `df.rename(columns={"Preço": "preco"})` e a coluna continuou com o nome antigo. Cite duas causas possíveis.
3. A coluna `cep` foi lida como número decimal (01310.0). Qual deve ser o tipo correto e por quê?
4. A data 05/03/2024 pode ser 5 de março ou 3 de maio. Como garantir que o pandas leia no formato dia/mês/ano?
5. Uma coluna numérica e assimétrica tem 45% de valores ausentes. Que opções você consideraria e que cuidados tomaria?
6. Escreva o código com `.loc` para criar a coluna `porte`: “Pequeno” se `area` $\leq$ 50, “Médio” se ficar entre 50 e 150 (inclusive), e “Grande” se for maior que 150.
7. Em que situação você prefere *Equal Frequency* a *Equal Width*? Dê um exemplo.
8. Por que não se deve criar dummies para uma variável ordinal como `faixa_preco`? E por que não para `suburb` (314 categorias)?
9. O que acontece se você exportar com `df.to_csv("base.csv")` (sem `index=False`) e depois reler o arquivo?

Respostas dos exercícios

1. `pd.read_csv("arquivo.csv", sep=";", decimal=",", encoding="latin-1").`
2. (a) Faltou reatribuir o resultado: `df = df.rename(...)`. (b) O nome do dicionário não corresponde exatamente ao da coluna (acento, maiúscula, espaço): o pandas ignora a chave sem avisar. Para ser avisado, use `errors="raise"`.
3. **Texto (string)**. O CEP é um código, não uma quantidade: não se faz conta com ele, e o zero à esquerda (01310) se perderia como número.
4. `pd.to_datetime(df["data"], format="%d/%m/%Y")` (ou `dayfirst=True`). Depois confira o período coberto e procure datas com primeiro número maior que 12.
5. Opções: imputar por mediana (global ou por grupo), criar categoria/indicador de ausência, ou excluir a coluna se ela não for essencial. Cuidados: usar mediana (a média é sensível a assimetria), medir o impacto, evitar descartar linhas em excesso e **registrar como limitação** que quase metade dos valores é estimada.
6.

```
df["porte"] = ""
df.loc[df["area"] <= 50, "porte"] = "Pequeno"
df.loc[(df["area"] > 50) & (df["area"] <= 150), "porte"] = "Médio"
df.loc[df["area"] > 150, "porte"] = "Grande"
```
7. Quando a distribuição é muito assimétrica e se quer grupos de tamanho comparável. Exemplo: renda de clientes, em que poucos ganham muito mais; Equal Width colocaria quase todos na primeira faixa.
8. Ordinais têm ordem, e as dummies a perderiam; devem ser codificadas respeitando a sequência (1, 2, 3, 4). *suburb* geraria 314 colunas, tornando a base pesada e difícil de interpretar; escolha uma variável com poucas categorias.
9. Uma coluna extra com a numeração das linhas é gravada; ao reler, ela aparece como *Unnamed: 0* e polui a base.

— Fim do guia —